

# From Program Logics Towards Language Logics

Matteo Cimini  
University of Massachusetts Lowell

November 26th, 2025  
ICTAC 2025, Marrakech, Morocco

# Language Verification

type safe?  
strongly normalizing?  
noninterference holds?

Language  
Definition

Language  
Analysis

Expression  $e ::= | x | \lambda x : T.e | e e$

...

$(\lambda x : T.e) v \longrightarrow e[v/x]$

if true then  $e_1$  else  $e_2 \longrightarrow e_1$

if false then  $e_1$  else  $e_2 \longrightarrow e_2$

...

# What Language Definitions?

|            |                                                                                                                                                          |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Type       | $T ::= \text{Int} \mid \text{Float} \mid T \rightarrow T \mid \text{Ref } T \mid \text{Unit}$                                                            |
| Expression | $e ::= v \mid e \oplus e \mid x \mid e e \mid \text{ref } e \mid !e \mid e := e \mid e; e \mid \text{error} \mid \text{try } e \text{ with } e$          |
| Value      | $v ::= n \mid f \mid \lambda x : T. e \mid \ell \mid \text{unit}$                                                                                        |
| Error      | $er ::= \text{error}$                                                                                                                                    |
| EvalCtx    | $E ::= \square \mid E \oplus e \mid v \oplus E \mid E e \mid \text{ref } E \mid !E \mid E := e \mid v := E \mid E; e \mid \text{try } E \text{ with } e$ |
| ErrorCtx   | $F ::= \square \mid F \oplus e \mid v \oplus F \mid F e \mid \text{ref } F \mid !F \mid F := e \mid v := F \mid F; e \mid \text{try } F \text{ with } e$ |

Type System

$$\boxed{\Sigma \mid \Gamma \vdash e : T}$$

Reduction Semantics

$$\boxed{e, \mu \longrightarrow e, \mu}$$

$$\begin{array}{c}
 \frac{\Sigma \mid \Gamma \vdash e : \text{Ref } T}{\Sigma \mid \Gamma \vdash !e : T} \quad
 \begin{array}{l}
 (\lambda x : T. e) v, \mu \longrightarrow e[v/x], \mu \\
 !\ell, \mu \longrightarrow \mu(\ell); \mu \\
 \ell := v, \mu \longrightarrow \text{unit}; [\ell \mapsto v] \mu \\
 v; e, \mu \longrightarrow e, \mu \\
 \text{try } v \text{ with } e, \mu \longrightarrow v, \mu \\
 \text{try error with } e, \mu \longrightarrow e, \mu
 \end{array} \\
 \\
 \frac{\Sigma \mid \Gamma \vdash e_1 : \text{Ref } T \quad \Sigma \mid \Gamma \vdash e_2 : T}{\Sigma \mid \Gamma \vdash e_1 := e_2 : \text{Unit}} \quad
 \frac{e, \mu \longrightarrow e', \mu'}{E[e], \mu \longrightarrow E[e'], \mu'} \quad
 \frac{}{F[er], \mu \longrightarrow er, \mu} [\text{ERR-CTX}]
 \end{array}$$

## Program Verification

pen&paper proofs  
interactive theorem proving  
automated proving  
type systems

program logics

model checking  
control flow analysis  
abstract interpretation

...

## Language Verification

pen&paper proofs  
interactive theorem proving  
automated proving  
type systems

program logics

**Question:**

*Can we apply program logics to Language Verification?*



# Contributions: Language Logics

Pre-condition  
assertions

Post-condition  
assertions

$\{P\} \mathcal{X} \{Q\}$

Piece of operational semantics

$\mathcal{X}$  can be:

- Expression  $e ::= v \mid e \oplus e \mid x \mid e e \mid \mathbf{ref} \ e \mid !e \mid e := e \mid e; e$
- $$\frac{\Gamma \vdash e_1 : T_1 \rightarrow T_2 \quad \Gamma \vdash e_2 : T_1}{\Gamma \vdash e_1 e_2 : T_2}$$

# Contributions: Language Logics

Pre-condition  
assertions

Post-condition  
assertions

$$\{P\} \mathcal{X} \{Q\}$$

Piece of operational semantics

**Example:**  $\mathcal{X}$  is an inference rule  $r$

*“when  $P$  holds,  
 $Q$  holds after having added  $r$  to the language definition”*

# What Language Definitions?

|            |                                                                                                                                                          |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Type       | $T ::= \text{Int} \mid \text{Float} \mid T \rightarrow T \mid \text{Ref } T \mid \text{Unit}$                                                            |
| Expression | $e ::= v \mid e \oplus e \mid x \mid e e \mid \text{ref } e \mid !e \mid e := e \mid e; e \mid \text{error} \mid \text{try } e \text{ with } e$          |
| Value      | $v ::= n \mid f \mid \lambda x : T. e \mid \ell \mid \text{unit}$                                                                                        |
| Error      | $er ::= \text{error}$                                                                                                                                    |
| EvalCtx    | $E ::= \square \mid E \oplus e \mid v \oplus E \mid E e \mid \text{ref } E \mid !E \mid E := e \mid v := E \mid E; e \mid \text{try } E \text{ with } e$ |
| ErrorCtx   | $F ::= \square \mid F \oplus e \mid v \oplus F \mid F e \mid \text{ref } F \mid !F \mid F := e \mid v := F \mid F; e \mid \text{try } F \text{ with } e$ |

## Mistake: Evaluator can ignore the error handler

$\text{try } 2 \div 0 \text{ with } \ell := !\ell + 1 \longrightarrow \text{try error with } \ell := !\ell + 1$   
 $\longrightarrow \text{error}$

Type

$$\boxed{\Sigma \mid \Gamma \vdash e : T}$$

Reduction Semantics

$$\boxed{e, \mu \longrightarrow e, \mu}$$

$$\begin{array}{c}
 \frac{\Sigma \mid \Gamma \vdash e_1 : \text{Ref } T \quad \Sigma \mid \Gamma \vdash e_2 : T}{\Sigma \mid \Gamma \vdash e_1 := e_2 : \text{Unit}} \quad
 \begin{array}{l}
 !\ell, \mu \longrightarrow \mu(\ell); \mu \\
 \ell := v, \mu \longrightarrow \text{unit}; [\ell \mapsto v] \mu \\
 v; e, \mu \longrightarrow e, \mu \\
 \text{try } v \text{ with } e, \mu \longrightarrow v, \mu \\
 \text{try error with } e, \mu \longrightarrow e, \mu \quad \text{X}
 \end{array}
 \end{array}$$

$$\frac{e, \mu \longrightarrow e', \mu'}{E[e], \mu \longrightarrow E[e'], \mu'}$$

$$\boxed{
 \begin{array}{l}
 [\text{ERR-CTX}] \\
 F[er], \mu \longrightarrow er, \mu
 \end{array}
 }$$

# Language Logic $\mathbb{L}$ - Base logic

$$\begin{array}{c}
 \text{(LANG)} \\
 \frac{\begin{array}{c} \{P\} \ G \ \{Q\} \\ \{Q\} \ I \ \{R\} \end{array}}{\{P\} \ (G, I) \ \{R\}}
 \end{array}
 \qquad
 \begin{array}{c}
 \text{(GRAM)} \\
 \frac{\begin{array}{c} \{P_0\} \ g_1 \ \{P_1\} \\ \{P_1\} \ g_2 \ \{P_2\} \\ \dots \\ \{P_{n-1}\} \ g_n \ \{P_n\} \end{array}}{\{P_0\} \ g_1 \ \dots \ g_n \ \{P_n\}}
 \end{array}
 \qquad
 \begin{array}{c}
 \text{(INF)} \\
 \frac{\begin{array}{c} \{P_0\} \ r_1 \ \{P_1\} \\ \{P_1\} \ r_2 \ \{P_2\} \\ \dots \\ \{P_{n-1}\} \ r_n \ \{P_n\} \end{array}}{\{P_0\} \ r_1 \ \dots \ r_n \ \{r_n\}}
 \end{array}$$

$$\begin{array}{c}
 \text{(PERM-G)} \\
 \frac{\begin{array}{c} \pi \text{ is a permutation of } g_1 \ \dots \ g_n \\ \{P\} \ \pi \ \{Q\} \end{array}}{\{P\} \ g_1 \ \dots \ g_n \ \{Q\}}
 \end{array}
 \qquad
 \begin{array}{c}
 \text{(PERM-R)} \\
 \frac{\begin{array}{c} \pi \text{ is a permutation of } r_1 \ \dots \ r_n \\ \{P\} \ \pi \ \{Q\} \end{array}}{\{P\} \ r_1 \ \dots \ r_n \ \{Q\}}
 \end{array}$$

$$\begin{array}{c}
 \text{(\mathcal{X}\text{-NEUTRAL})} \\
 \{P\} \ \mathcal{X} \ \{P\}
 \end{array}
 \qquad
 \begin{array}{c}
 \text{(CONSEQUENCE)} \\
 \frac{P \Rightarrow P' \quad \{P'\} \ \mathcal{X} \ \{Q'\} \quad Q' \Rightarrow Q}{\{P\} \ \mathcal{X} \ \{Q\}}
 \end{array}$$

$$\begin{array}{c}
 \text{(ITERATE)} \\
 \frac{\{P\} \ \mathcal{X} \ \{Q\} \quad \{Q\} \ \mathcal{X} \ \{R\}}{\{P\} \ \mathcal{X} \ \{R\}}
 \end{array}$$

# Language Logic + Theory - $\mathbb{L}(\mathcal{T})$

$P ::= \text{inductive}(X, c, \{n_1, \dots, n_k\}) \mid \text{handles-error}(op, n)$

(HANDLES-ERROR)

$$\frac{\text{inductive}(F, op, I) \in P \quad er \Rightarrow_G^* t_i \quad i \notin I}{\left[ \frac{\{P\} \quad \tilde{f}}{(op \ t_1, \dots, t_n), \tilde{s} \longrightarrow t, \tilde{s}'} \right] \{P \wedge \text{handles-error}(op, i)\}}$$

**Example of deriving inductive:**

$\{ \} \quad \text{Type } T ::= \dots \mid T \longrightarrow T \mid \dots \quad \{ \text{inductive}(T, \longrightarrow, \{1,2\}) \}$

$\{ \} \quad \text{ErrorCtx } F ::= \dots \mid \text{try } F \ e \mid \dots \quad \{ \text{inductive}(F, \text{try}, \{1\}) \}$

# Example of Application of Language Logics

$$\begin{array}{c}
 \text{(HANDLES-ERROR)} \\
 \frac{\text{inductive}(F, op, I) \in P \quad er \Rightarrow_G^* t_i \quad i \notin I}{\{P\} \left[ \frac{\tilde{f}}{(op \ t_1, \dots, t_n), \tilde{s} \longrightarrow t, \tilde{s}'} \right] \{P \wedge \text{handles-error}(op, i)\}}
 \end{array}$$

Applied to buggy lang: (with **ErrorCtx**  $F ::= \dots \mid \text{try } F \ e \mid \dots$  )

$$\begin{array}{c}
 \text{(HANDLES-ERROR)} \\
 \frac{\text{inductive}(F, try, \{1\}) \in P \quad er \Rightarrow_G^* \text{error} \quad \boxed{1 \notin \{1\}}}{\{P\} \text{ try error with } e, s \longrightarrow e, s \{ \dots \}}
 \end{array}$$

Applied to fixed lang: (with no **Error Ctx** for **F** )

$$\begin{array}{c}
 \text{(HANDLES-ERROR)} \\
 \frac{\text{inductive}(F, try, \{\}) \in P \quad er \Rightarrow_G^* \text{error} \quad \boxed{1 \notin \{\}}}{\{P\} \text{ try error with } e, s \longrightarrow e, s \{P \wedge \text{handles-error}(try, 1)\}}
 \end{array}$$



# Automated Prover: Lang-n-Assert

```
./lنا TheoryOfErrorHandlers  
'true' lambda_ref_err_bad.lان 'HandlesError(try,1)'
```

**Proof not found:**

*Argument number 1 of the operator **try** is subject to an error context and may not handle the error.*

```
./lنا TheoryOfErrorHandlers  
'true' lambda_ref_err_fixed.lان 'HandlesError(try,1)'
```

**Proof found:**

...

...

---

grammar

---

inference rules

---

```
'true' lambda_ref_err_fixed.lان 'HandlesError(try,1)'
```

## In the paper

- Formalism for language logics and three theories
- Example of analysis on a faulty language
- When faulty language fixed: Proof derivations

## In the tool repo

- Implementation of the automated prover
- Other examples

## Future Work

- Soundness of the language logic
- Extending the language logic for full type safety, noninterference, strong normalization, etc.

**Thank you!**

